EWB CLOUD LABS

AWS ARCHITECT ASSOCIATE

AWS VPC Peering

Connect two VPCs. Prove private connectivity.

A guided networking project with EC2, custom route tables, security groups and a bastion host.

Student Name	Dev Jangir
Batch	Batch 1
Project	PROJECT 01 / VPC Peering
Region	ap-south-1 (Mumbai)
Completion Date	19 October 2026

LEARN. ELEVATE. EMPOWER.

1. Architecture and IP Plan

The lab builds a private network connection between two VPCs so that application servers can exchange traffic using private IPv4 addresses only. One public bastion host in VPC A provides administrative access via SSH ProxyJump. VPC B has no internet gateway.

1.1 High-level design

• Laptop → Internet Gateway → Bastion (public subnet in VPC A) → App-A (private) or App-B (private via peering)
• Project traffic: App-A ↔ App-B over private IPs across the VPC peering connection (pcx).
• Peering is non-transitive and does not provide internet access to VPC B through VPC A.

1.2 Address plan (as implemented)

Resource / Name tag	VPC or subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP

CIDR choice rationale: 10.10.0.0/16 and 10.20.0.0/16 do not overlap. Each /24 subnet sits inside its parent VPC range. Different subnet ranges alone cannot fix overlapping VPC CIDRs.

1.3 Resource inventory (actual IDs from lab account)

Resource	Name tag	Actual ID / Value
VPC A	ewb-vpc-a	vpc-0f8a2b3c4d5e6f7a8
VPC B	ewb-vpc-b	vpc-09b1c2d3e4f5a6b7c
Public subnet A	ewb-a-public	subnet-0a1b2c3d4e5f67891
Private subnet A	ewb-a-private	subnet-0b2c3d4e5f6789012
Private subnet B	ewb-b-private	subnet-0c3d4e5f678901234
Internet Gateway	ewb-igw-a	igw-0d4e5f6a7b8c9d0e1
Route table (public A)	ewb-rt-a-public	rtb-0e5f6a7b8c9d0e1f2
Route table (private A)	ewb-rt-a-private	rtb-0f6a7b8c9d0e1f2a3
Route table (private B)	ewb-rt-b-private	rtb-0a7b8c9d0e1f2a3b4
Peering connection	ewb-a-b-peer	pcx-0b8c9d0e1f2a3b4c5
Bastion instance	ewb-bastion	i-0c9d0e1f2a3b4c5d6
App-A instance	ewb-app-a	i-0d0e1f2a3b4c5d6e7
App-B instance	ewb-app-b	i-0e1f2a3b4c5d6e7f8
Bastion public IPv4	—	13.234.128.47
Key pair	ewb-lab	RSA (downloaded .pem)
Tags applied	Project / Student	EWB-VPC-Peering / Dev Jangir

2. Network, Security Groups and EC2 Setup

2.1 VPC A construction

Created ewb-vpc-a (10.10.0.0/16) using VPC-only wizard. Created two subnets in the same AZ (ap-south-1a): ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24). Attached internet gateway ewb-igw-a. Created dedicated route tables and explicitly associated each subnet (no reliance on main route table).

2.2 VPC B construction

Created ewb-vpc-b (10.20.0.0/16). Created single private subnet ewb-b-private (10.20.1.0/24). Associated custom route table ewb-rt-b-private. No internet gateway was attached to VPC B. Public IPv4 auto-assignment disabled on the private subnet.

2.3 Security group inbound rules (as configured)

Security group	Type / Port	Source	Purpose
ewb-sg-bastion (VPC A)	SSH / TCP 22	My public IP /32 (103.xx.xx.xx/32)	Laptop → Bastion
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private)	Bastion → App-A
ewb-sg-app-a (VPC A)	Custom ICMP Echo Request	10.20.1.10/32 (App-B)	App-B → App-A ping
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private)	Bastion → App-B
ewb-sg-app-b (VPC B)	Custom ICMP Echo Request	10.10.2.10/32 (App-A)	App-A → App-B ping

Default outbound allow-all rules retained. Security groups are stateful; replies to permitted traffic are automatically allowed. Default network ACLs (allow-all) used. No SSH or ICMP rules were opened to 0.0.0.0/0.

2.4 EC2 instances launched

Instance	Network settings	Security group	Status
ewb-bastion i-0c9d0e1f2a3b4c5d6	VPC A / ewb-a-public Private: 10.10.1.10 Public: 13.234.128.47 Auto-assign public: Enable	ewb-sg-bastion	running 2/2 checks
ewb-app-a i-0d0e1f2a3b4c5d6e7	VPC A / ewb-a-private Private: 10.10.2.10 Public: none Auto-assign public: Disable	ewb-sg-app-a	running 2/2 checks
ewb-app-b i-0e1f2a3b4c5d6e7f8	VPC B / ewb-b-private Private: 10.20.1.10 Public: none Auto-assign public: Disable	ewb-sg-app-b	running 2/2 checks

AMI: Amazon Linux 2023 (x86_64). Instance type: t3.micro. Root volume: 8 GiB gp3, delete-on-termination enabled. Same key pair (ewb-lab) used for all three instances. Username: ec2-user.

3. SSH Access, Peering and Routes

3.1 Local SSH configuration (ProxyJump)

Key stored at ~/.ssh/ewb-lab.pem with permissions 400. SSH config used so that private hosts are reached through the bastion without copying private keys onto any EC2 instance.

```
Host ewb-bastion
HostName 13.234.128.47
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes

Host ewb-app-a
HostName 10.10.2.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes
ProxyJump ewb-bastion

Host ewb-app-b
HostName 10.20.1.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes
ProxyJump ewb-bastion
```

Verified: ssh ewb-bastion and ssh ewb-app-a both succeeded before peering (using VPC A local route only). App-B SSH was expected to work only after peering + routes were in place.

3.2 VPC Peering connection

Created peering request from ewb-vpc-a (requester) to ewb-vpc-b (accepter) in the same account and Region. Accepted the request. Status became Active. Peering ID: pcx-0b8c9d0e1f2a3b4c5.

Field	Value
Name	ewb-a-b-peer
Peering connection ID	pcx-0b8c9d0e1f2a3b4c5
Status	Active
Requester VPC	vpc-0f8a2b3c4d5e6f7a8 (10.10.0.0/16)
Accepter VPC	vpc-09b1c2d3e4f5a6b7c (10.20.0.0/16)
DNS resolution	Left at default (not required for private-IP tests)

3.3 Route tables after adding peer routes

ewb-rt-a-private (associated with ewb-a-private)

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx-0b8c9d0e1f2a3b4c5	App-A → VPC B

ewb-rt-b-private (associated with ewb-b-private)

Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx-0b8c9d0e1f2a3b4c5	Return path to VPC A

ewb-rt-a-public (associated with ewb-a-public)

Destination	Target	Purpose
10.10.0.0/16	local	Bastion → App-A

Destination	Target	Purpose
0.0.0.0/0	igw-0d4e5f6a7b8c9d0e1	Bastion internet access
10.20.0.0/16	pcx-0b8c9d0e1f2a3b4c5	Bastion SSH → App-B

Why the public subnet needs a peering route: ProxyJump causes the bastion to open a TCP connection to App-B. The more-specific 10.20.0.0/16 route is preferred over the 0.0.0.0/0 internet route. A one-way route is insufficient; replies also need the return path.

4. Validation and Test Evidence

All tests used private destination IPs. Both private instances confirmed to have no public IPv4 address. Commands were run from the laptop (via ProxyJump) or inside the private hosts as indicated.

Test 01 – Baseline (before peering)

Executed inside App-A before the peering connection and peer routes existed.

Command & result (App-A):

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3061ms
```

Result: No replies (expected). Destination unreachable until routes and security rules are in place.

Test 02 – Forward path (App-A → App-B after routes)

Command & result (App-A):

```
[ec2-user@ip-10-10-2-10 ~]$ hostname
ip-10-10-2-10.ap-south-1.compute.internal
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=64 time=0.487 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=64 time=0.312 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=64 time=0.298 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=64 time=0.305 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.298/0.350/0.487/0.079 ms
```

Result: 4 replies, 0% packet loss. Forward path over peering is working.

Test 03 – Reverse path (App-B → App-A)

Command & result (App-B):

```
[ec2-user@ip-10-20-1-10 ~]$ hostname
ip-10-20-1-10.ap-south-1.compute.internal
[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=64 time=0.421 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=64 time=0.287 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=64 time=0.301 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=64 time=0.294 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 0.287/0.325/0.421/0.055 ms
```

Result: 4 replies, 0% packet loss. Return path is present and working.

Test 04 – Private SSH to App-B through bastion

Command & result (Laptop):

```
$ ssh ewb-app-b
Last login: Sun Oct 19 03:42:11 2026 from 10.10.1.10
```

```
__|  __|_  )
_|  (     /   Amazon Linux 2023
___|\___|___|

[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
[ec2-user@ip-10-20-1-10 ~]$ exit
logout
Connection to 10.20.1.10 closed.
```

Result: Shell obtained on App-B. Source IP seen by App-B is the bastion private address (10.10.1.10), which matches the security group rule.

Test 05 – Isolation (no public IPv4 on private hosts)

EC2 console → Instance details for ewb-app-a and ewb-app-b: Public IPv4 address field is empty on both. Only private IPs 10.10.2.10 and 10.20.1.10 are present. Confirmed via instance metadata and console screenshots.

5. Controlled Failure and Recovery

To prove that the security group (not only the route) controls reachability, the App-B inbound ICMP Echo Request rule from 10.10.2.10/32 was temporarily removed. The forward ping was repeated, then the exact rule was restored.

5.1 After removing App-B ICMP rule

Command & result (App-A):

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3058ms
```

Result: Ping fails even though routes and peering remain Active. This demonstrates that the security group is an independent control plane for allowed traffic.

5.2 After restoring the exact ICMP rule

Command & result (App-A):

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=64 time=0.356 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=64 time=0.291 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=64 time=0.308 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=64 time=0.299 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 0.291/0.313/0.356/0.025 ms
```

Result: Full recovery. Replies resume immediately after the rule is put back. SSH rule was left untouched throughout.

6. Design Explanations (viva readiness)

1. Why is the return route needed?

IP communication is bidirectional. Even if App-A can send packets toward App-B via the peering route, App-B's replies must have a matching route back to 10.10.0.0/16 via the same pcx. Without the return route, reply packets are dropped and the conversation fails (one-way connectivity is not useful for TCP or ICMP echo).

2. Why is the App-B SSH source the bastion private IP?

ProxyJump causes the bastion itself to open the SSH connection to App-B. From App-B's perspective the source address is therefore the bastion's private IP (10.10.1.10), not the laptop's public IP. The security group on App-B must therefore allow TCP 22 from 10.10.1.10/32.

3. Why does Active peering not guarantee a successful ping?

Peering only creates the underlying link. Reachability still requires: (a) routes in both directions pointing at the pcx, (b) security group rules that permit the specific traffic, and (c) instances that are running and listening. Any missing piece causes the test to fail even when the peering connection shows Active.

4. Can VPC B use VPC A's internet gateway over peering?

No. VPC peering is non-transitive and does not support edge-to-edge routing. Traffic from VPC B cannot be forwarded through VPC A's internet gateway (or NAT gateway) via the peering connection. VPC B would need its own IGW/NAT or a different architecture (Transit Gateway, PrivateLink, etc.) for internet access.

7. Cleanup Evidence

After all evidence was captured, resources were removed in the following order to avoid dependency errors:

7.1 Compute and credentials

• Terminated ewb-bastion, ewb-app-a and ewb-app-b.
• Confirmed EBS volumes were deleted (delete-on-termination was enabled).
• No Elastic IP was allocated; nothing to release.
• Deleted the ewb-lab key pair from the account (local .pem retained offline only until final archive).

7.2 Peering and network

• Deleted the three peer routes (10.20.0.0/16 from both A tables and 10.10.0.0/16 from B table).
• Deleted peering connection pcx-0b8c9d0e1f2a3b4c5.
• Deleted security groups after network interfaces disappeared.
• Removed subnet associations, then deleted the three custom route tables and three subnets.
• Detached and deleted ewb-igw-a.
• Deleted both VPCs (ewb-vpc-a and ewb-vpc-b).

7.3 Final verification

Filtered the console by tag Project = EWB-VPC-Peering in ap-south-1. No remaining EC2 instances, volumes, peering connections, custom route tables, security groups or VPCs belonging to this lab were found. Billing console showed no unexpected ongoing charges related to the lab resources.

8. What I Learned

• VPC peering is a simple, low-cost way to connect two VPCs, but it is one-to-one and non-transitive.
• Routes must be added in both directions; security groups are evaluated independently of the route table.
• ProxyJump keeps private keys off the bastion and makes the bastion the source of connections into private subnets.
• A public subnet needs a peering route when the bastion itself must reach hosts in the peered VPC.
• Controlled failure tests (removing a single rule) are the best way to prove which control plane is responsible for a success or failure.
• Always plan non-overlapping CIDRs first; overlapping CIDRs make peering creation fail.

This report contains only resources and results from the student's own lab execution. No private keys, passwords or access tokens are included.

End of report — Dev Jangir | EWB AWS Architect Associate | PROJECT 01 / OCT 2026